

FoodRescue

Connecting Surplus Food to People Who Need It

Project Documentation

Chingu Voyage — 6-Week Build

June 2026

Stack: Node.js · React · PostgreSQL · Socket.io · Leaflet.js

1. Product Requirements Document (PRD)

1.1 Overview

Product Name	FoodRescue
Version	1.0 — MVP
Timeline	6 Weeks (Chingu Voyage)
Team Size	4–5 Developers
Stack	Node.js + React + PostgreSQL

1.2 Problem Statement

Restaurants, bakeries, and food vendors routinely discard surplus food at end-of-day due to no viable redistribution channel. Meanwhile, individuals and communities nearby go without adequate food access. FoodRescue solves this by creating a real-time platform that connects food donors with claimers before expiry windows close.

1.3 Goals & Objectives

- Enable food donors (restaurants, bakeries, vendors) to list surplus food in under 2 minutes
- Allow claimers to discover, claim, and pick up food before it expires
- Reduce local food waste by creating a frictionless redistribution loop
- Build public accountability through an impact metrics dashboard
- Deliver a fully functional, demo-ready product within 6 weeks

1.4 Target Users

Donors

- Restaurants with end-of-day surplus
- Bakeries with unsold baked goods
- Caterers with post-event leftovers
- Grocery stores with near-expiry produce

Claimers

- Individuals looking for free or reduced-price food
- Community organisations and food banks
- Students and low-income households

1.5 Feature Prioritisation

Feature	Description	Priority
User Authentication	Donor & Claimer roles, JWT-based login/register	🔴 Must Have
Create Listing	Food name, photo, qty, location, expiry, pickup window	🔴 Must Have
Browse Feed	Card-based feed with distance filter and countdown timers	🔴 Must Have
Claim Flow	One-click claim, confirmation modal, auto-status update	🔴 Must Have
Donor Dashboard	Manage active/past listings, mark as picked up	🔴 Must Have
Claimer Dashboard	View claimed items and pickup history	🔴 Must Have
Live Map View	Leaflet.js map showing all active listings by location	🟡 Should Have
Real-time Updates	Socket.io listing disappears instantly when claimed	🟡 Should Have
Expiry Countdown	Ticking timer on each listing card	🟡 Should Have
Rating System	Claimers rate donors after successful pickup	🟡 Should Have
AI Categorisation	Claude API auto-tags food type from listing text	🟢 Nice to Have
Impact Stats Page	Public counter kg rescued, meals saved	🟢 Nice to Have
Email Notifications	Notify nearby claimers when new food is listed	🟢 Nice to Have

1.6 User Stories

Donor

1. As a donor, I want to create a food listing with a photo and expiry time so that claimers know what is available and for how long.
2. As a donor, I want to view all my active listings so that I can track what has been claimed.
3. As a donor, I want to mark a listing as picked up after a claimer collects the food.
4. As a donor, I want to edit or delete a listing if plans change.

Claimer

5. As a claimer, I want to browse nearby food listings so that I can find available food close to me.
6. As a claimer, I want to claim a listing in one click so that I can secure it before someone else does.

7. As a claimer, I want to see a countdown timer on each listing so that I know how urgently I need to act.
8. As a claimer, I want to rate a donor after pickup so that the community can trust listing quality.

1.7 Out of Scope (v1.0)

- Payment processing or monetary transactions
- Mobile native app (iOS/Android)
- Multi-language support
- Admin moderation panel
- Social sharing / referral system

2. Technical Requirements Document (TRD)

2.1 Tech Stack

Layer	Technology	Rationale
Frontend	React	Fast builds, no SSR overhead needed for MVP
Styling	Tailwind CSS	Rapid UI development, consistent design tokens
Backend	Node.js + Express	Team comfort zone, robust REST API support
Database	PostgreSQL + Prisma	Relational model fits; Prisma accelerates dev
Authentication	JWT + bcrypt	Stateless auth, no vendor lock-in
Real-time	Socket.io	Claim updates and live feed refresh
Maps	Leaflet.js	Free, no billing surprises, great React support
File Uploads	Cloudinary	Free tier sufficient, CDN delivery, image transforms
AI (optional)	Anthropic Claude API	Auto food categorisation and smart suggestions
Frontend Deploy	Vercel	Git-integrated, free, instant previews
Backend Deploy	supbase	Simple Node deploys, managed PostgreSQL available

2.2 Database Schema

Users

Field	Type	Notes
id	UUID (PK)	Auto-generated
name	VARCHAR(100)	Display name
email	VARCHAR(255)	Unique, used for login
passwordHash	TEXT	bcrypt hashed
role	ENUM	donor claimer
location	POINT (PostGIS)	Lat/Lng for proximity queries
createdAt	TIMESTAMP	Auto-set on creation

Listings

Field	Type	Notes
id	UUID (PK)	Auto-generated
donorId	UUID (FK → Users)	The posting donor
title	VARCHAR(150)	e.g. 'Jollof Rice & Chicken'
category	VARCHAR(50)	Auto-tagged or manually set
description	TEXT	Optional extra details
photoUrl	TEXT	Cloudinary CDN URL
quantity	VARCHAR(50)	e.g. '20 portions'
location	POINT (PostGIS)	Pickup coordinates
address	TEXT	Human-readable address
expiresAt	TIMESTAMP	When food must be claimed by
pickupStart	TIMESTAMP	Pickup window start
pickupEnd	TIMESTAMP	Pickup window end
status	ENUM	active claimed expired completed
createdAt	TIMESTAMP	Auto-set

Claims & Ratings

Field	Type	Notes
id (Claims)	UUID (PK)	
listingId	UUID (FK → Listings)	The listing being claimed
claimerId	UUID (FK → Users)	The claimer
claimedAt	TIMESTAMP	Auto-set
status	ENUM	pending confirmed no-show
id (Ratings)	UUID (PK)	
claimId	UUID (FK → Claims)	One rating per claim
score	INT (1–5)	Star rating
comment	TEXT	Optional feedback

2.3 API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register new user (donor or claimer)

POST	/api/auth/login	Login and return JWT
GET	/api/listings	Get all active listings (with filters: distance, category)
POST	/api/listings	Create a new listing (donor only)
GET	/api/listings/:id	Get single listing detail
PATCH	/api/listings/:id	Update listing (donor only)
DELETE	/api/listings/:id	Delete listing (donor only)
POST	/api/listings/:id/claim	Claim a listing (claimer only)
PATCH	/api/listings/:id/pickup	Mark as picked up (donor only)
POST	/api/claims/:id/rate	Submit rating after pickup (claimer only)
GET	/api/users/me/listings	Donor get own listings
GET	/api/users/me/claims	Claimer get own claims
GET	/api/stats/impact	Public impact counter (kg rescued, meals saved)

2.4 Non-Functional Requirements

- API response time under 300ms for all listing fetch endpoints
- Images compressed and served via Cloudinary CDN — max upload size 5MB
- JWT token expiry: 7 days; refresh strategy to be implemented in v1.1
- All API routes protected except GET /listings and GET /stats/impact
- Socket.io events scoped to listing rooms to avoid unnecessary broadcasts
- PostgreSQL queries use indexed columns: donorId, status, expiresAt, location
- Application deployable via environment variables only — no hardcoded secrets

3. App Flow

3.1 Onboarding Flow

New users land on the public feed (read-only). A persistent banner prompts sign-up. Registration requires: name, email, password, role selection (Donor or Claimer), and location permission.

9. User visits FoodRescue.app
10. Views public listing feed in read-only mode
11. Clicks 'Sign Up' → selects role: Donor or Claimer
12. Enters name, email, password → location permission prompt
13. Account created → redirected to relevant dashboard

3.2 Donor Flow

Creating a Listing

14. Donor logs in → lands on Donor Dashboard
15. Clicks 'Post Food' → multi-step form opens
16. Step 1: Food details — title, description, category, quantity
17. Step 2: Upload photo (Cloudinary) — required
18. Step 3: Pickup details — address, pickup window start/end, expiry time
19. Submits → listing goes live immediately on the public feed
20. Socket.io broadcasts new listing to all connected clients

Managing Listings

21. Donor Dashboard shows: Active, Claimed, Completed, Expired tabs
22. Active listings show a live countdown to expiry
23. Claimed listings show claimer name and estimated pickup time
24. Donor clicks 'Mark as Picked Up' after claimer collects food
25. Status updates to 'Completed' → claimer prompted to leave rating

3.3 Claimer Flow

Discovering Food

26. Claimer logs in → lands on Browse Feed (card view default)
27. Feed shows listings sorted by distance + expiry urgency
28. Claimer can toggle to Map View — pins on Leaflet map
29. Filter options: distance radius, food category, pickup time
30. Each card shows: photo, title, qty, distance, expiry countdown, donor rating

Claiming Food

31. Claimer clicks a listing card → detail modal opens
32. Reviews full description, pickup address, and pickup window
33. Clicks 'Claim This Food' → confirmation prompt appears
34. On confirm: listing status → 'Claimed'; removed from public feed instantly
35. Claimer Dashboard updates with new claimed item
36. Donor receives real-time notification via Socket.io

Post-Pickup

37. After donor marks pickup as complete, claimer gets notification
38. Claimer visits dashboard → 'Rate your experience' prompt
39. Submits 1–5 star rating with optional comment
40. Donor's average rating updates on their profile and listings

3.4 Real-time Events (Socket.io)

Event	Emitted By	Received By
listing:new	Server (on POST /listings)	All connected clients
listing:claimed	Server (on POST /claim)	All clients viewing that listing
listing:expired	Server (cron job)	Donor of the listing
listing:completed	Server (on PATCH /pickup)	Claimer of the listing
listing:updated	Server (on PATCH /listing)	All connected clients

3.5 Error & Edge Case Handling

- If a user tries to claim an already-claimed listing → error toast: 'Sorry, this was just claimed'
- If expiry time passes before claim → listing auto-moves to 'Expired' via cron job (runs every 5 minutes)
- If donor deletes active listing → any claimer in progress gets notified
- Location permission denied → manual address entry fallback for listing creation
- Cloudinary upload failure → form shows retry prompt, listing not submitted

4. UI/UX Design Brief

4.1 Design Philosophy

FoodRescue should feel warm, trustworthy, and urgent — like a community bulletin board that's also a countdown clock. The UI must work equally well on mobile (primary use case for claimers on the go) and desktop (donor management).

4.2 Brand Identity

Product Name	FoodRescue
Tagline	Don't waste it. Share it.
Tone	Warm, community-driven, action-oriented
Personality	Approachable, trustworthy, slightly urgent

4.3 Colour Palette

Role	Hex	Usage
Primary Green	#2D6A4F	Buttons, headings, active states
Light Green	#D8F3DC	Card backgrounds, highlights, badges
Accent Green	#74C69D	Hover states, dividers, icons
Urgency Orange	#F4845F	Expiry countdowns under 1 hour
Danger Red	#E63946	Expired listings, error states
Dark	#1B1B1B	Body text, headings
Mid Gray	#6B7280	Secondary text, metadata
Light Gray	#F3F4F6	Page background, table stripes
White	#FFFFFF	Card surfaces, modals

4.4 Typography

Primary Font	Inter (Google Fonts)
Heading Weight	Bold (700)
Body Weight	Regular (400) / Medium (500)
Base Size	16px
Heading Scale	H1: 32px H2: 24px H3: 20px

4.5 Key Screens

1. Browse Feed (Claimer Home)

- Full-width listing cards in a responsive 2–3 column grid
- Each card: food photo (top), title, category badge, quantity, distance, donor star rating, expiry countdown (colour shifts orange < 1hr, red < 15min)
- Sticky top bar: search input, category filter chips, distance slider, toggle between Feed / Map views
- 'Claim' button on each card — triggers confirmation modal inline

2. Map View

- Full-screen Leaflet.js map behind a floating panel
- Green pins for available listings; orange for expiring soon; grey for claimed
- Click pin → mini popup card with photo, title, countdown, Claim button
- Floating card list scrolls alongside the map

3. Create Listing (Donor)

- 3-step wizard with progress indicator at top
- Step 1: Food info — title, description, category dropdown, quantity
- Step 2: Photo upload — drag-and-drop zone with preview
- Step 3: Pickup logistics — address autocomplete, time pickers for pickup window and expiry
- Live preview card on right side (desktop) shows how the listing will appear

4. Donor Dashboard

- Summary stats row: Active listings, Total claimed, Total completed, Average rating
- Tab navigation: Active | Claimed | Completed | Expired
- Each listing row: thumbnail, title, status badge, time remaining, action buttons

5. Claimer Dashboard

- Summary: Total claimed, Meals this month, Favourite donors
- Tab navigation: Upcoming Pickups | History
- Each row: food photo, title, donor name + rating, pickup address, time, status

4.6 Component Library

- ListingCard — reused across Feed, Map popup, and Dashboards
- CountdownBadge — colour-shifting timer component
- CategoryBadge — coloured pill with icon (e.g. Baked Goods, Cooked Meals, Produce)
- ClaimModal — full-screen overlay on mobile, centred modal on desktop
- RatingStars — interactive 1–5 star input and display

- ImpactCounter — animated counter for public stats page

4.7 Responsive Breakpoints

Mobile (< 640px)	Single column feed, bottom nav bar, full-screen map
Tablet (640–1024px)	Two-column feed, side navigation
Desktop (> 1024px)	Three-column feed, live preview on forms, full dashboard

4.8 Accessibility

- All interactive elements keyboard navigable
- Countdown timer announces changes to screen readers via aria-live
- Colour is never the sole indicator of state — always paired with text or icon
- Minimum contrast ratio 4.5:1 on all body text
- All images have descriptive alt text

5. Implementation Plan

5.1 Team Roles

Dev	Role	Primary Responsibilities
Dev 1	Backend Lead	Auth system, API architecture, Prisma schema, middleware
Dev 2	Backend / Data	Listings CRUD, Cloudinary integration, cron jobs, DB queries
Dev 3	Frontend Lead	Feed UI, listing cards, countdown component, design system
Dev 4	Frontend / Real-time	Map view (Leaflet), Socket.io client, dashboards
Dev 5	Full-stack / DevOps	Ratings, AI integration, deployment (Vercel + Railway), CI

5.2 6-Week Sprint Plan

Week 1 — Foundation

Goal: Repo setup, auth working, database connected

- Initialise monorepo structure (client/ + server/ or separate repos)
- Configure ESLint, Prettier, Husky pre-commit hooks
- Set up PostgreSQL + Prisma — define and migrate schema (Users, Listings, Claims, Ratings)
- Implement POST /auth/register and POST /auth/login with JWT
- Build auth pages: Register (with role selection), Login, basic protected route
- Configure Cloudinary account and test image upload endpoint
- Deploy skeleton app to Vercel + Railway — CI/CD from main branch
- Team alignment: agree on component naming, git branching strategy, PR review process

Week 2 — Core Listings

Goal: Donors can post, claimers can browse

- Build Create Listing API (POST /api/listings) with Cloudinary photo upload
- Build GET /api/listings with distance filter (PostGIS ST_DWithin)
- Build Listing Detail endpoint (GET /api/listings/:id)
- Frontend: Create Listing 3-step wizard with live preview
- Frontend: Browse Feed — responsive card grid with CountdownBadge
- Frontend: Listing Detail modal
- End-of-week: Donor can post food; claimer can see it in the feed

Week 3 — Claim Flow & Dashboards

Goal: Full claim loop works end-to-end

- Build POST `/api/listings/:id/claim` endpoint (status lock — prevent double claim)
- Build PATCH `/api/listings/:id/pickup` endpoint (donor marks complete)
- Implement cron job — auto-expire listings past `expiresAt` every 5 minutes
- Frontend: Claim flow — button → confirmation modal → success state
- Frontend: Donor Dashboard (tabs: Active, Claimed, Completed, Expired)
- Frontend: Claimer Dashboard (tabs: Upcoming, History)
- End-of-week: Full claim cycle testable with two browser tabs

Week 4 — Real-time & Map

Goal: Live updates, map view, rating system

- Integrate Socket.io — server emits `listing:new`, `listing:claimed`, `listing:expired`
- Frontend: Socket.io client hooks — feed updates without page refresh
- Frontend: Leaflet.js map view — colour-coded pins, click-to-claim popup
- Build POST `/api/claims/:id/rate` endpoint
- Frontend: RatingStars component — post-pickup rating prompt
- Display donor average rating on listing cards
- End-of-week: Real-time demo impressive enough for Chingu mid-voyage check-in

Week 5 — Polish & Impact

Goal: Production-quality UI, impact stats, edge cases

- Build GET `/api/stats/impact` endpoint — aggregate kg rescued, meals saved, donors active
- Frontend: Public Impact Stats page with animated counters
- UI polish pass — mobile responsiveness audit across all screens
- Error handling — empty states, loading skeletons, toast notifications
- Form validation — client-side + server-side
- Edge cases: expired listing claim attempt, duplicate claim guard, missing location fallback
- (Optional) Claude API integration — auto-categorise listing on creation

Week 6 — Testing, Demo Prep & Launch

Goal: Ship it. Demo it. Be proud of it.

- End-to-end testing — full donor → claimer → rating cycle
- Seed database with realistic demo data (at least 10 listings, 3 donors, 5 claimers)
- Performance audit — Lighthouse score > 85 on Feed and Dashboard
- Security review — JWT validation, input sanitisation, rate limiting on auth endpoints
- Write README — setup instructions, env vars, architecture diagram
- Record demo video (2–3 min) showing donor posting, claimer claiming, live map
- Final deployment to production URLs — share with Chingu evaluators

5.3 Git Branching Strategy

main	Production-ready code only. Protected — no direct pushes.
dev	Integration branch. All features merged here first.
feature/*	Individual features. e.g. feature/listing-crud, feature/map-view
fix/*	Bug fixes. e.g. fix/expiry-cron
chore/*	Tooling, config, docs. e.g. chore/ci-setup

5.4 Definition of Done

A feature is 'done' when all of the following are true:

41. Code is reviewed and approved by at least one other team member
42. No TypeScript/linting errors in CI
43. Feature works on mobile and desktop viewports
44. API endpoint returns correct status codes for happy path and error cases
45. Merged to dev branch and deployment preview is green

5.5 Risk Register

Risk	Likelihood	Mitigation
PostGIS setup complexity	Medium	Use simple lat/lng + Haversine formula as fallback if PostGIS unavailable
Socket.io scaling issues	Low	Scope events to listing rooms; defer scaling to v1.1
Team availability drop in Week 5	Medium	Front-load core features; Week 5–6 are polish/buffer
Cloudinary free tier limits	Low	Compress images client-side before upload; 25GB free is ample for demo
Scope creep on AI features	High	AI is explicitly optional — timebox to 4 hours or cut